

HTML5 & CSS3 Car Rental Website

Learn by Doing: Semantic HTML5 + Flexbox + CSS Grid

1. Learning Objectives

By completing this exercise, students should be able to:

1. Construct a webpage using **semantic HTML5 elements**.
 2. Organize content using `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, and `<footer>`.
 3. Apply CSS3 selectors and properties to style a webpage.
 4. Use **Flexbox** for one-dimensional layouts such as navigation menus and cards.
 5. Use **CSS Grid** for two-dimensional layouts such as car/product displays.
 6. Create a responsive webpage structure.
 7. Understand the separation between **content (HTML)** and **presentation (CSS)**.
-

2. Website We Are Going to Build

Our fictional company will be called:

DriveEasy Car Rental

The website will contain:

LOGO	Reservation	Rent	Return	Payment	About Us
				Login /	Signup
HERO SECTION					
Rent Your Perfect Car Today					
[Search / Book a Car]					
PROMOTIONS					
Weekend Deal Long-Term Rental New Customer					
OUR CARS					
[Car 1]	[Car 2]	[Car 3]	[Car 4]		
SOCIAL PROOF					
"Great service!"		"Easy booking!"			
★★★★★		★★★★★			
FOOTER					
About Contact Terms Privacy Social					

This gives students an opportunity to use **both Flexbox and Grid for different layout problems.**

3. Project Structure

Ask students to create the following folder:

```
car-rental-website/  
├── index.html  
├── css/  
│   └── style.css  
├── images/  
│   ├── hero-car.jpg  
│   ├── car1.jpg  
│   ├── car2.jpg  
│   ├── car3.jpg  
│   └── car4.jpg
```

└─ README.md

For the first exercise, however, students can work with only:

index.html
css/style.css

4. Part 1 — Create the HTML5 Skeleton

Create a file called:

index.html

Start with the basic HTML5 document structure.

Starter Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">

  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

  <title>DriveEasy Car Rental</title>

  <link rel="stylesheet" href="css/style.css">
</head>

<body>

  <!-- Website content will be added here -->

</body>

</html>
```

5. Understanding the HTML5 Skeleton

```
<!DOCTYPE html>

<!DOCTYPE html>
```

This tells the browser that the document uses **HTML5**.

<html>

```
<html lang="en">
```

The `<html>` element is the root element of the webpage.

The `lang` attribute specifies the language.

```
lang="en"
```

means English.

<head>

The `<head>` contains information about the webpage that is generally not displayed directly as page content.

For example:

```
<meta charset="UTF-8">
```

allows the browser to correctly display many characters.

Responsive viewport

```
<meta name="viewport"  
      content="width=device-width, initial-scale=1.0">
```

This is particularly important for responsive web design.

It tells mobile browsers to use the device's actual screen width.

External CSS

```
<link rel="stylesheet" href="css/style.css">
```

This connects our HTML document to our CSS file.

This demonstrates an important principle:

HTML controls structure and meaning. CSS controls presentation and layout.

6. Part 2 — Build the Header

Now we start building the website.

Replace the comment inside `<body>` with:

```
<header>

  <div class="logo">
    DriveEasy
  </div>

  <nav>
    <ul>
      <li><a href="#">Reservation</a></li>
      <li><a href="#">Rent</a></li>
      <li><a href="#">Return</a></li>
      <li><a href="#">Payment</a></li>
      <li><a href="#">About Us</a></li>
      <li><a href="#">Login / Signup</a></li>
      <li><a href="#">Logout</a></li>
    </ul>
  </nav>

</header>
```

7. Why Use `<header>`?

Instead of:

```
<div class="header">
```

we use:

```
<header>
```

because HTML5 provides **semantic elements**.

Semantic elements communicate the meaning and purpose of content.

Some important HTML5 semantic elements are:

Element	Purpose
<code><header></code>	Introductory/header content
<code><nav></code>	Navigation links
<code><main></code>	Main content
<code><section></code>	Major content section
<code><article></code>	Independent content
<code><aside></code>	Related/sidebar content
<code><footer></code>	Footer information

This improves:

- readability
- accessibility
- maintainability
- search-engine understanding

8. Part 3 — Build the Main Content

Now add the `<main>` element after `</header>`.

```
<main>

  <!-- Hero Section -->

  <section class="hero">

    <div class="hero-content">

      <h1>Rent Your Perfect Car</h1>

      <p>
        Easy, affordable and reliable car rental
        for every journey.
      </p>

      <a href="#" class="btn">
        Book a Car
      </a>

    </div>

  </section>

  <!-- Promotion Section -->
```

Tutorial 1 Coding Exercise

```
<section class="promotions">

  <h2>Special Promotions</h2>

  <div class="promotion-container">

    <article class="promotion-card">
      <h3>Weekend Special</h3>

      <p>
        Enjoy 20% off weekend rentals.
      </p>

      <a href="#">Learn More</a>
    </article>

    <article class="promotion-card">
      <h3>Long-Term Rental</h3>

      <p>
        Save more when you rent for
        more than 7 days.
      </p>

      <a href="#">Learn More</a>
    </article>

    <article class="promotion-card">
      <h3>New Customer</h3>

      <p>
        Get a special discount on your
        first rental.
      </p>

      <a href="#">Learn More</a>
    </article>

  </div>
</section>

<!-- Product / Car Section -->

<section class="cars">

  <h2>Our Cars</h2>

  <div class="car-grid">

    <article class="car-card">
```

Tutorial 1 Coding Exercise

```


<h3>Economy Car</h3>

<p>From $45 / day</p>

<a href="#" class="btn">
  Rent Now
</a>

</article>

<article class="car-card">

  <h3>Comfort Sedan</h3>

  <p>From $65 / day</p>

  <a href="#" class="btn">
    Rent Now
  </a>

</article>

<article class="car-card">

  <h3>Family SUV</h3>

  <p>From $85 / day</p>

  <a href="#" class="btn">
    Rent Now
  </a>

</article>

<article class="car-card">

  <h3>Luxury Car</h3>

  <p>From $120 / day</p>

  <a href="#" class="btn">
```

Tutorial 1 Coding Exercise

```
        Rent Now
    </a>

</article>

</div>

</section>

<!-- Social Proof Section -->

<section class="testimonials">

    <h2>What Our Customers Say</h2>

    <div class="testimonial-container">

        <article class="testimonial">

            <p>
                "The booking process was simple
                and the car was excellent."
            </p>

            <h3>Sarah Tan</h3>

            <p>★★★★★</p>

        </article>

        <article class="testimonial">

            <p>
                "Great service and very friendly
                staff. I will rent again."
            </p>

            <h3>David Lim</h3>

            <p>★★★★★</p>

        </article>

        <article class="testimonial">

            <p>
                "The prices were reasonable and
                the whole process was easy."
            </p>

            <h3>Michael Lee</h3>
```

```
        <p>★★★★★</p>

    </article>

</div>

</section>

</main>
```

9. Part 4 — Build the Footer

After `</main>`, add:

```
<footer>

    <div class="footer-container">

        <div>
            <h3>DriveEasy</h3>

            <p>
                Your trusted partner for
                convenient car rental.
            </p>
        </div>

        <div>
            <h3>Quick Links</h3>

            <ul>
                <li><a href="#">Reservation</a></li>
                <li><a href="#">Rent</a></li>
                <li><a href="#">Return</a></li>
                <li><a href="#">Payment</a></li>
            </ul>
        </div>

        <div>
            <h3>Company</h3>

            <ul>
                <li><a href="#">About Us</a></li>
                <li><a href="#">Contact Us</a></li>
                <li><a href="#">Privacy Policy</a></li>
                <li><a href="#">Terms & Conditions</a></li>
            </ul>
        </div>
    </div>
```

```
        <div>
            <h3>Follow Us</h3>

            <p>
                Facebook | Instagram | LinkedIn
            </p>
        </div>

    </div>

    <div class="copyright">

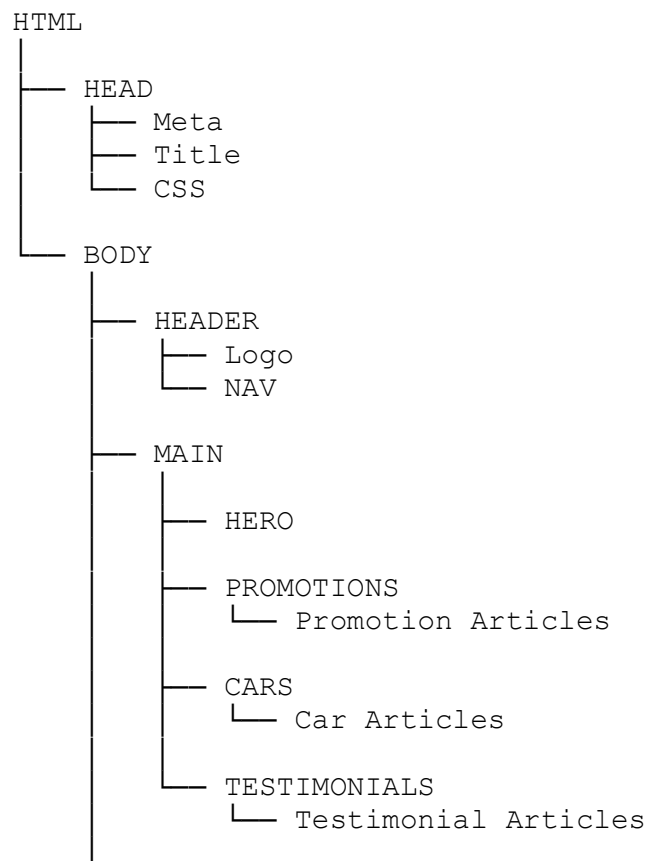
        <p>
            &copy; 2026 DriveEasy Car Rental.
            All rights reserved.
        </p>

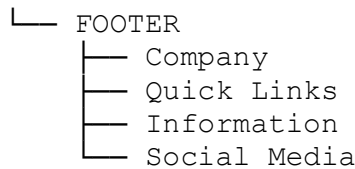
    </div>

</footer>
```

10. Complete HTML Structure

At this point, students should have this overall structure:





This is an important point for students:

Before writing CSS, understand the HTML structure.

CSS becomes much easier when students understand the document hierarchy.

11. Part 5 — Create the CSS File

Create:

`css/style.css`

Start with the following CSS skeleton.

```
/* =====
GLOBAL STYLES
===== */

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, Helvetica, sans-serif;
  line-height: 1.6;
  color: #333;
  background-color: #ffffff;
}

a {
  text-decoration: none;
}

img {
  width: 100%;
  display: block;
}

/* =====
HEADER
```

Tutorial 1 Coding Exercise

```
===== */

header {
  /* TODO: Use Flexbox */
}

/* =====
   LOGO
   ===== */

.logo {
  /* TODO */
}

/* =====
   NAVIGATION
   ===== */

nav ul {
  /* TODO: Use Flexbox */
}

nav li {
  /* TODO */
}

nav a {
  /* TODO */
}

nav a:hover {
  /* TODO */
}

/* =====
   HERO
   ===== */

.hero {
  /* TODO */
}

.hero-content {
  /* TODO */
}

.hero h1 {
  /* TODO */
}

.hero p {
  /* TODO */
}
```

Tutorial 1 Coding Exercise

```
/* =====  
  BUTTON  
  ===== */  
  
.btn {  
  /* TODO */  
}  
  
.btn:hover {  
  /* TODO */  
}  
  
/* =====  
  PROMOTIONS  
  ===== */  
  
.promotions {  
  /* TODO */  
}  
  
.promotions h2 {  
  /* TODO */  
}  
  
.promotion-container {  
  /* TODO: Use Flexbox */  
}  
  
.promotion-card {  
  /* TODO */  
}  
  
/* =====  
  CARS  
  ===== */  
  
.cars {  
  /* TODO */  
}  
  
.cars h2 {  
  /* TODO */  
}  
  
.car-grid {  
  /* TODO: Use CSS Grid */  
}  
  
.car-card {  
  /* TODO */  
}
```

Tutorial 1 Coding Exercise

```
.car-card h3 {
    /* TODO */
}

.car-card p {
    /* TODO */
}

/* =====
TESTIMONIALS
===== */

.testimonials {
    /* TODO */
}

.testimonials h2 {
    /* TODO */
}

.testimonial-container {
    /* TODO: Use Flexbox */
}

.testimonial {
    /* TODO */
}

/* =====
FOOTER
===== */

footer {
    /* TODO */
}

.footer-container {
    /* TODO: Use Grid */
}

footer ul {
    /* TODO */
}

footer li {
    /* TODO */
}

footer a {
    /* TODO */
}

.copyright {
    /* TODO */
}
```

```
}

/* =====
RESPONSIVE DESIGN
===== */

@media (max-width: 768px) {

  /* TODO:
    Modify the layout for tablets/mobile
  */

}
```

12. Student Challenge 1 — Style the Header

The first challenge is to make the logo and navigation appear **horizontally**.

The desired result is:

```
DriveEasy      Reservation Rent Return Payment About Us Login/Signup
Logout
```

Hint

Use:

```
display: flex;
```

on the `<header>`.

Students should discover that Flexbox can control the horizontal arrangement.

A possible solution is:

```
header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 60px;
}
```

Then:

```
nav ul {
```

```
display: flex;
list-style: none;
gap: 20px;
}
```

Key concepts

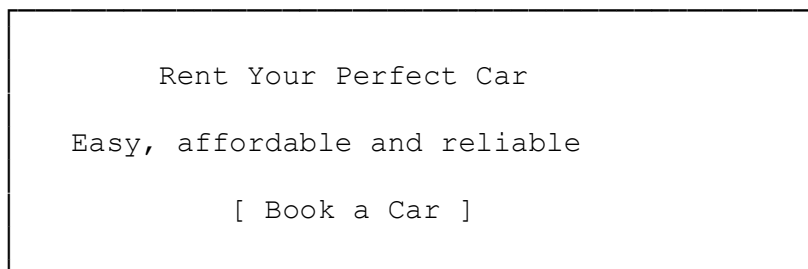
Students should understand:

```
display: flex;
justify-content: space-between;
align-items: center;
gap: 20px;
```

13. Student Challenge 2 — Style the Hero

The hero should become the visual focal point of the website.

For example:



Students should experiment with:

```
.hero {
  min-height: 500px;

  display: flex;
  align-items: center;
  justify-content: center;

  text-align: center;
}
```

This introduces an important Flexbox pattern:

```
display: flex;
align-items: center;
justify-content: center;
```

which can center content both vertically and horizontally.

14. Student Challenge 3 — Promotion Cards

We want three cards:

Weekend	Long-Term	New Customer
20% OFF	Save More	1st Rental

Use **Flexbox**.

```
.promotion-container {  
  display: flex;  
  gap: 20px;  
}
```

Students should then experiment with:

```
.promotion-card {  
  flex: 1;  
}
```

What does `flex: 1` do?

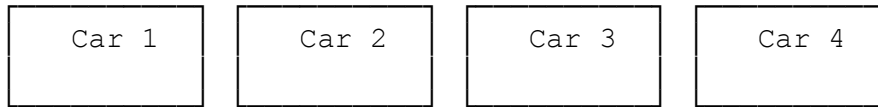
It allows the cards to share the available space.

This is a useful practical demonstration of how Flexbox distributes space.

15. Student Challenge 4 — Car Product Grid

Now we deliberately switch from Flexbox to **CSS Grid**.

The desired layout:



Use:

```
.car-grid {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 25px;  
}
```

This introduces:

`grid-template-columns`

and:

`1fr`

What is 1fr?

`fr` means **fraction of the available space**.

For example:

```
grid-template-columns: repeat(4, 1fr);
```

means:

Create four columns that share the available space equally.

16. Flexbox vs Grid

This exercise provides an excellent opportunity to teach students the difference.

Flexbox	CSS Grid
One-dimensional	Two-dimensional
Row OR column	Rows AND columns
Navigation	Product gallery
Button groups	Card layouts

Flexbox	CSS Grid
Simple alignment	Complex page layouts
Content-driven	Layout-driven

A useful teaching principle:

Use Flexbox when you primarily need to arrange items along one direction.
Use Grid when you need control over rows and columns.

17. Student Challenge 5 — Make the Car Cards Attractive

Students can experiment with:

```
.car-card {  
  border: 1px solid #ddd;  
  border-radius: 10px;  
  overflow: hidden;  
  padding-bottom: 20px;  
}
```

Then:

```
.car-card h3,  
.car-card p {  
  margin: 10px 20px;  
}
```

And:

```
.car-card .btn {  
  display: inline-block;  
  margin-left: 20px;  
}
```

This demonstrates the relationship between:

- box model
- padding
- margin
- border
- width

- height
- content

18. Student Challenge 6 — Social Proof

The testimonial section can use Flexbox:

```
.testimonial-container {  
  display: flex;  
  gap: 20px;  
}
```

Each testimonial:

```
.testimonial {  
  flex: 1;  
  padding: 25px;  
  border-radius: 10px;  
}
```

Students should recognize that **social proof** is not just a design element; it represents content commonly found on commercial websites.

Examples include:

- customer testimonials
- ratings
- reviews
- number of customers
- trust badges
- partner logos

19. Student Challenge 7 — Footer Grid

The footer is another good opportunity to use CSS Grid.

```
.footer-container {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 30px;  
}
```

Students can compare this with the earlier Flexbox examples.

20. Responsive Design Challenge

Finally, students should make the website responsive.

On a desktop:

```
[Car] [Car] [Car] [Car]
```

On a tablet:

```
[Car] [Car]  
[Car] [Car]
```

On a mobile phone:

```
[Car]  
[Car]  
[Car]  
[Car]
```

Students can use:

```
@media (max-width: 768px) {  
  
    .car-grid {  
        grid-template-columns: repeat(2, 1fr);  
    }  
  
}
```

Then for smaller screens:

```
@media (max-width: 480px) {  
  
    .car-grid {  
        grid-template-columns: 1fr;  
    }  
  
}
```

This introduces the concept of **responsive breakpoints**.

21. Recommended Learning Sequence

I would teach this exercise in the following order rather than giving students the completed CSS immediately:

Stage 1 — HTML Structure

Students create:

```
HTML5
↓
header
↓
nav
↓
main
↓
section
↓
article
↓
footer
```

Stage 2 — Basic CSS

Students learn:

```
Selectors
↓
Colors
↓
Fonts
↓
Margins
↓
Padding
↓
Borders
```

Stage 3 — Flexbox

Students solve:

```
Header
Navigation
Promotion cards
Testimonials
```

Stage 4 — CSS Grid

Students solve:

Car product gallery
Footer

Stage 5 — Responsive Design

Students solve:

Desktop
↓
Tablet
↓
Mobile

22. Final Student Exercise

After completing the tutorial, give students this challenge:

You are a frontend developer working for DriveEasy Car Rental. Your task is to design and implement a responsive car-rental website using HTML5 and CSS3.

Requirements

The website must contain:

1. Header

- Company logo
- Navigation menu
- Reservation
- Rent
- Return
- Payment
- About Us
- Login/Signup
- Logout

2. Hero

- Main heading
- Description
- Call-to-action button

3. Promotions

- Minimum 3 promotional cards

4. Cars

- Minimum 4 cars
- Car image
- Car name
- Rental price
- Rent button

5. Social Proof

- Minimum 3 customer testimonials
- Customer names
- Ratings

6. Footer

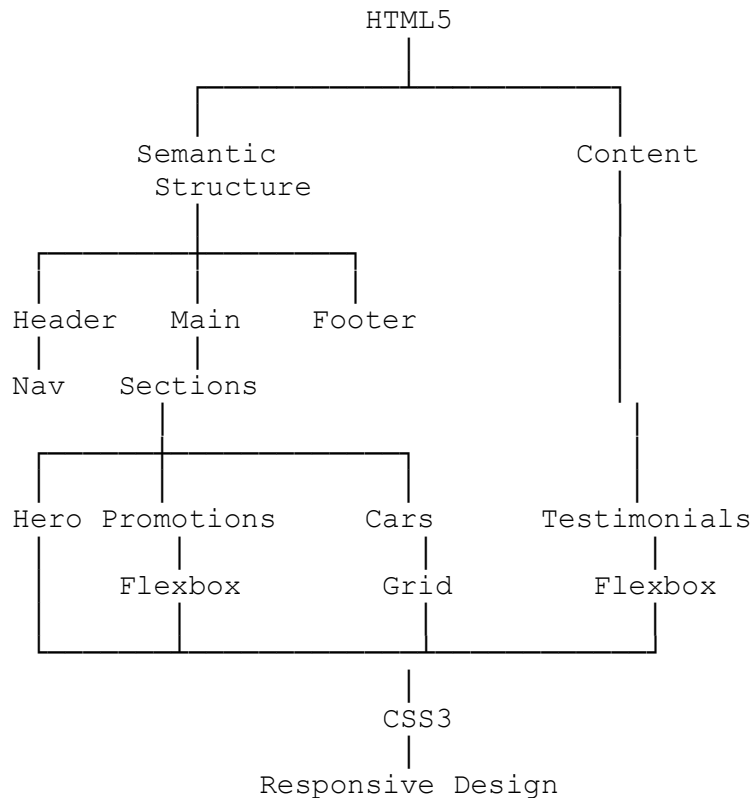
- Company information
- Quick links
- Social media
- Copyright

7. CSS Requirements

- CSS selectors
 - Box model
 - Flexbox
 - CSS Grid
 - Hover effects
 - Responsive media queries
-

23. Suggested Final Architecture

The final webpage should demonstrate this architecture:



Learning Point

Construct the page progressively:

HTML structure → Basic CSS → Flexbox problem → Grid problem → Responsive problem → Refinement

That turns the exercise from a "copy this website" activity into an actual **frontend programming problem-solving exercise**.

A particularly useful pedagogical distinction for this tutorial is:

"What am I trying to arrange?"

- Logo + navigation → **Flexbox**
- Promotion cards → **Flexbox**
- Car catalogue → **CSS Grid**
- Testimonials → **Flexbox**
- Footer columns → **CSS Grid**